

eMICRO: Real-Time Multi-Hop Access Control for Microservices with eBPF

Rizky Ramadhana
Putra
Virginia Tech
Blacksburg, VA, USA
rizky@vt.edu

Osama Bajaber
King Abdulaziz University
Jeddah, Saudi Arabia
obajaber@kau.edu.sa

Saimon Amanuel Tsegai
Virginia Tech
Blacksburg, VA, USA
saimon.tsegai@vt.edu

Teryl Taylor
IBM Research
Yorktown Heights, NY, USA
terylt@ibm.com

Frederico Araujo
IBM Research
Yorktown Heights, NY, USA
frederico.araujo@ibm.com

Yuede Ji
UT Arlington
Arlington, TX, USA
yuede.ji@uta.edu

Peng Gao
Virginia Tech
Blacksburg, VA, USA
penggao@vt.edu

Abstract

Modern cloud applications often comprise thousands of microservices whose interactions form complex request paths. Traditional inter-service access control restricts individual service-to-service requests, but fails to prevent *multi-hop attacks*, where each hop appears legitimate yet the overall path violates security intent. This gap leaves systems exposed to unauthorized access and data exfiltration. In this paper, we present eMICRO, a path-aware defense system for microservices that prevents such attacks while remaining efficient and deployable. eMICRO enforces real-time multi-hop access control through three key techniques: (1) history-based access control extended to capture service invocation sequences; (2) security policies encoded as efficient deterministic finite automaton (DFA), supporting constant-time lookups and compact label propagation; (3) eBPF-based in-kernel request tracing for transparent, low-overhead enforcement without code changes. Evaluations on DeathStarBench and production cloud traces from Uber, Alibaba, and ByteDance, covering 12 million request workflows and thousands of services, demonstrate the scalability of eMICRO. eMICRO performs policy checks in 1 μ s, stores 50 million policies in only 100 MB, and reduces propagation overhead by 90% with negligible run-time impact. These results show that eMICRO delivers scalable and efficient protection against multi-hop attacks, making it practical for deployment in large-scale microservice environments.

CCS Concepts

• **Security and privacy** → **Network security**; **Distributed systems security**; **Information flow control**.

Keywords

Microservices, Access Control, Multi-Hop Attacks, eBPF

ACM Reference Format:

Rizky Ramadhana Putra, Osama Bajaber, Saimon Amanuel Tsegai, Teryl Taylor, Frederico Araujo, Yuede Ji, and Peng Gao. 2026. eMICRO: Real-Time

Multi-Hop Access Control for Microservices with eBPF. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3832688>

1 Introduction

Microservices decompose modern applications into loosely coupled services that communicate over the network. At scale, such systems may consist of thousands of services, producing millions of interdependent requests and intricate request sequences. This complexity enlarges the attack surface by introducing opportunities for *multi-hop attacks* [22], in which adversaries exploit legitimate inter-service dependencies to construct request chains that are syntactically valid but semantically violate security intentions. This *generic threat* manifests in diverse forms across modern microservice deployments. For example, Datadog reported cryptojacking campaigns in Docker Swarm and Kubernetes [43], where attackers first compromised unauthenticated Internet-facing services and then abused service dependencies to hijack resources for cryptocurrency mining.

Existing defenses largely decouple inter-service authorization from intra-service enforcement. Intra-service defenses (e.g., system call specialization [20, 27]) constrain the execution of individual services but ignore requests exchanged across services, and thus cannot associate execution spanning multiple services. Inter-service defenses (e.g., network policies and inter-service access control [2, 9, 13, 38, 39, 44, 49, 50]) regulate communication between pairs of services while overlooking each service's internal execution, and are unable to capture the causal dependencies needed to connect a chain of service interactions. Therefore, neither class can effectively prevent multi-hop attacks. Provenance-based defenses for microservices [23, 29, 46] can reconstruct complete request sequences, but their use is largely limited to post-attack forensic analysis rather than real-time prevention. Information-flow-based defenses [17, 40] extend visibility beyond adjacent hops, but do not capture the precise ordering of request sequences.

Early approaches toward real-time multi-hop enforcement [22, 28, 42, 47] suffer from scalability limitations. They rely on *centralized servers* that introduce bottlenecks and single points of failure, incur millisecond-level delays, and do not support concurrent requests efficiently. Many also depend on specific frameworks or



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '26, The Hague, Netherlands*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2871-6/2026/11

<https://doi.org/10.1145/3830454.3832688>

libraries, resulting in ad hoc solutions. Consequently, the problem of preventing multi-hop attacks in real time, while ensuring scalability, efficiency, and deployability, has not been fully addressed.

Goals and challenges. To mitigate multi-hop attacks in microservices, we aim to design a *real-time defense* that enforces policies over *entire* request sequences rather than individual hops. Achieving this goal requires overcoming three key challenges.

First, *Preventing multi-hop attacks requires validating the entire invocation history up to the originating service*, since intermediate hops often encode critical security intent. However, naively enumerating all permissible request sequences quickly leads to policy explosion, as real deployments exhibit a combinatorial number of workflows, making policies brittle and difficult to maintain. The challenge is to support policies that can concisely constrain request paths, such as requiring, forbidding, or matching specific intermediate services, without sacrificing administrative usability.

Second, *enforcing multi-hop policies must be real-time and scalable*. This requires the enforcement mechanism to track the sequence of preceding requests while remaining lightweight enough for real-time operation. Naively attaching the complete preceding request sequences to each request and sequentially checking them against all policies incurs $O(n)$ time complexity in the number of policies, and does not scale when thousands of access control decisions are performed per second. Production cloud traces [32, 33, 35, 37, 41] report deployments with up to 40,000 services and request paths of length up to 80, yielding large per-request context.

Prior multi-hop microservice access control approaches [22, 34, 47] rely on a centralized remote controller to reconstruct and evaluate request histories across the cluster. This adds a network round-trip on every hop and creates bottlenecks and a single point of failure. A natural attempt is to replicate the controller via a consensus protocol like Raft [45], but this does not remove the bottleneck. Raft makes the controller highly available, yet the cost comes from per-request evaluation over a causal chain whose events are produced at every upstream service. However the controller is replicated, requests tracing and policy enforcement must still reach the decision point, either through a per-hop round trip or through per-request Raft consensus, beyond the update rate Raft is designed for.

Modern orchestration platforms such as Kubernetes further complicate this challenge: services are dynamically scheduled, replicated, and migrated across physical machines, with underlying placement hidden behind service discovery, overlay networks, service meshes, and sidecars, so a service rarely knows which physical host its peer runs on. A user-space observer is often insufficient, since namespace isolation limits visibility and frequent user/kernel context switches add substantial overhead.

Third, *microservices are implemented using diverse languages, libraries, frameworks, and parallelism paradigms*. They communicate over heterogeneous protocols (e.g., HTTP, RPC) and middleware stacks (e.g., Thrift, gRPC, Avro). Propagating request-sequence context across workflows requires instrumenting applications or the communication libraries and frameworks they depend on, incurring substantial engineering overhead. This ecosystem is also large, rapidly evolving, and highly heterogeneous. For example, Meta reports production use of 16 programming languages [33], while Jaeger and OpenTelemetry [10] maintain instrumentation across

11 languages, each with multiple dialects (e.g., Meta's Hack). An effective solution therefore must propagate hop information and enforce policies in a *communication-stack-agnostic manner*, without requiring much application code modification.

Contributions. We present **EMicro**, a **decentralized multi-hop access control system for microservices** that secures entire request workflows by enforcing end-to-end policies. EMicro prevents multi-hop attacks in real time using a path- and order-aware policy language that is compiled into a compact DFA, distributed across services, enforced at line rate with minimal overhead, and requires no manual application or library instrumentation while remaining compatible with common cloud networking platforms. EMicro advances microservices security beyond inter-service single-hop access control [2, 9, 13, 44, 50] and information-flow-based defenses that do not enforce request order [17, 18, 40]. Compared with prior centralized multi-hop access control systems [22, 28, 34, 42, 47], EMicro's primary contribution lies in scalable, instrumentation-free, decentralized enforcement rather than new policy semantics.

Our contributions are threefold:

- (1) *Expressive multi-hop policy language.* EMicro introduces a policy language that models benign request workflows and prevents multi-hop attacks by reasoning about complete request paths rather than only adjacent service hops. Instead of enumerating all valid workflows, the language provides path-level predicates and logical operators that compactly capture security intent over intermediate services. By going beyond simple sequence matching, it avoids policy explosion while enabling rich security semantics. As a result, the language remains practical and easy for administrators to author and maintain.
- (2) *Compact DFA-based representation for multi-hop policies.* EMicro avoids propagating full request histories and sequentially checking policies, which does not scale and precludes real-time enforcement. Instead, EMicro compiles multi-hop policies describing valid request workflows into a single deterministic finite automaton (DFA) with *constant-time* complexity policy check, where each state represents a valid request sequence. *Requests propagate only the corresponding DFA state label*, rather than complete histories, yielding significantly smaller labels both theoretically and in practice. To support large policy sets, our compiler employs a data-parallel algorithm that merges thousands of policy DFAs efficiently using a tree-structured parallel reduction.
- (3) *Application-agnostic label propagation and policy enforcement via eBPF.* EMicro propagates DFA state labels and enforces multi-hop policies *transparently* without modifying application code. Leveraging eBPF technology [6], EMicro operates at the kernel level, enabling low-overhead label propagation across network packets, processes, threads, and coroutines, while avoiding user-kernel context switches. Policies are replicated across nodes using consensus protocols (Raft [45]), while enforcement remains local at each service and decentralized.

Evaluation. We conducted extensive evaluations on a 9-VM deployment and evaluated EMicro using large-scale production traces from Alibaba [32, 41], Uber [37], and ByteDance [35]. We also deployed EMicro on the DeathStarBench [26] testbed running 27

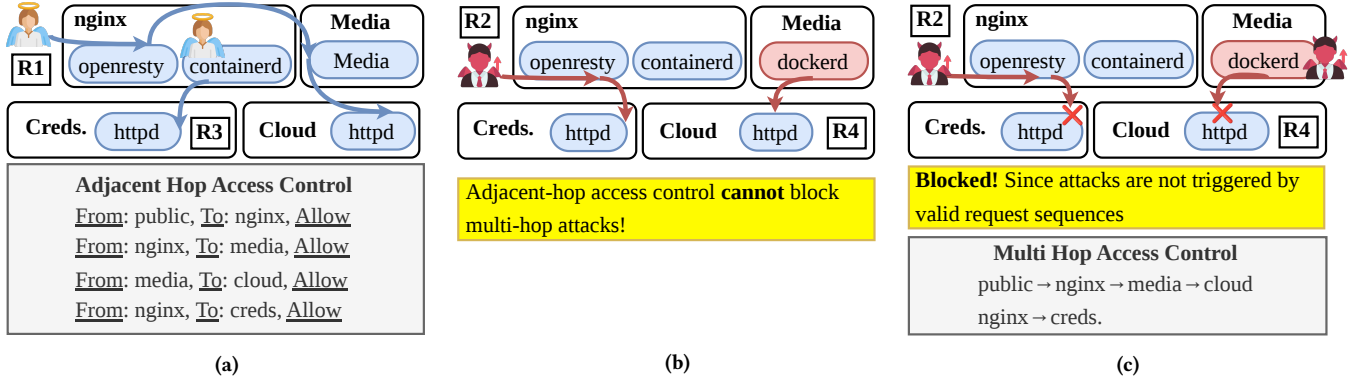


Fig. 1: Two valid request workflows (R1, R3). A developer uses a malicious Docker image that performs network enumeration (R2). An attacker exploits a server-side request forgery (SSRF) vulnerability in nginx to send requests to creds (R4). Both R3 and R4 send requests from nginx to creds, and both R1 and R2 send requests from media to cloud. Adjacent-hop access control treats these cases identically, enabling multi-hop attacks due to missing end-to-end context.

services with 10 replicas each and processing 12 million requests over 12 hours under realistic network conditions (latency, jitter, packet loss). The testbed reflects real-world microservice deployments, spanning six programming languages (Java, Node.js, Python, C/C++, JavaScript, Go), open-source services (NGINX, Memcached, MongoDB, Cylon, Xapian, MySQL, RabbitMQ), common protocols (HTTP, RPC), frameworks (Apache Thrift, gRPC), and custom user-defined applications. In addition to benign workloads, we evaluated representative multi-hop attack scenarios, including open-redirection attacks, remote code execution (RCE)-based workflow bending, and malicious container-based network enumeration. We also compared eMICRO with prior centralized multi-hop enforcement systems (CloudCover [22]) and information-flow-based defenses (Kalium [34] and Trapeze [17]).

Our evaluation results demonstrate that: (1) eMICRO can effectively block all malicious flows in real-time while incurring **less than 10 μ s per-request overhead**. (2) eMICRO's DFA-based compilation reduces the space required to propagate request sequences by 90% on average, requiring only ~ 100 MB to store 50 million policies with constant-time lookups under 1 μ s. (3) eMICRO's fully distributed enforcement achieves **35% shorter completion times** compared to CloudCover [22], and performs policy checks up to 10 $\times$ faster than Kalium [34] and Trapeze [17], incurring negligible overhead to legitimate traffic. (4) To further demonstrate eMICRO's robustness under failures, we integrate eMICRO with the Raft consensus protocol [45] for policy distribution. Our results show that policy updates reach quorum in under 4 seconds, and failed nodes recover missing updates within an additional 4 seconds, ensuring fast failover recovery and consistent policy enforcement across nodes. These results demonstrate that eMICRO effectively prevents multi-hop attacks while remaining efficiently deployable, non-intrusive, and minimally disruptive to benign workloads.

2 Background and Related Work

Microservices is an architectural style in which applications are decomposed into small, loosely coupled services that interact over the network using protocols such as HTTP or RPC. Each service

typically executes inside a container managed by a runtime such as Docker [5] or containerd [3]. While containers share the same underlying kernel, they maintain isolation in networking, file system, and processes through namespaces. Deployments at scale are orchestrated by platforms such as Docker Swarm [5] and Kubernetes [12], which establish virtual networking between services, commonly implemented through iptables or container network interface (CNI) plugins such as Cilium [2] and Calico [13].

2.1 Motivating Example

To illustrate the limitations in microservice access control mechanisms against multi-hop attacks, consider a web-based photo-sharing application composed of four services: an nginx frontend, an image processing service media, a cloud storage, and a credential management service creds. Fig. 1a shows the two legitimate workflows supported by the application. In the first workflow, a user loads a webpage containing a picture. The browser sends the request to nginx, which forwards it to the media. The media retrieves the image from cloud, applies transformations such as resizing and compression, and returns the processed image to the user via nginx. In the second workflow, the nginx frontend server periodically retrieves short-lived tokens from creds, which are used to securely access media and cloud without embedding long-term secrets.

Now suppose the nginx contains a server-side request forgery (SSRF) vulnerability. An attacker could exploit this flaw to redirect requests so that nginx forwards them to creds instead of cloud. This enables the attacker to extract authentication tokens and subsequently access other services. Conventional microservice access control considers only adjacent hops and cannot distinguish between requests *initiated directly* by the nginx and those *triggered indirectly via nginx*, as illustrated in Fig. 1b. Similarly, a compromised media container that performs network enumeration and sends requests to cloud cannot be prevented.

Preventing such attacks requires reasoning about entire request sequences, not just individual hops. For example, nginx should access creds only when doing so directly, not when acting on behalf of a user request. Similarly, media may access cloud only if

initiated by `[public,nginx]` request sequences. This approach, depicted in Fig. 1c, follows a history-based access control paradigm [16, 24, 25] in which actions are authorized based on their causal history rather than in isolation. In microservices, the relevant history corresponds to the sequence of requests recursively triggered to fulfill a workflow. Authorization decisions should therefore derive from the complete preceding request sequence.

2.2 Multi-Hop Attacks

Multi-hop attacks are categorized into two broad classes: path-suffix attacks, which exploit fragments of benign workflows, and workflow-bending attacks, which construct entirely new workflows that are not aligned with legitimate ones [22].

(i) Path-suffix attack. Consider again the photo-sharing application (Fig. 1). Two valid workflows exist: `[public,nginx,media,cloud]` and `[nginx,creds]`. Suppose an attacker compromises `media` and directly invokes `cloud`, creating the path `[media,cloud]`. Although this sequence resembles part of a benign workflow, it is invalid because it bypasses the required initiation from `public` and `nginx`. Path-suffix attacks are feasible in practice through software supply-chain vulnerabilities: malicious or trojanized container images may embed scripts that enumerate and contact other services. Prior work shows that more than 80% of images on Docker Hub contain at least one high-severity vulnerability [48], making this a significant attack vector.

(ii) Request-workflow-bending attack. Workflow-bending attacks deviate entirely from valid workflows [22]. In some cases, attackers exploit a benign service as a confused deputy. For example, an open-redirection vulnerability in `nginx` [4] could be abused to bend the workflow from `[public,nginx,media,cloud]` to `[public,nginx,creds]` (Fig. 1b). In other cases, attackers compromise a service, allowing them to inject malicious processes and generate arbitrary inter-service requests[7].

2.3 Microservices Access Control

Below, we review prior work on microservice access control and contrast it with our approach.

- *Adjacent-hop enforcement.* Approaches such as iptables, Cilium [2], Calico [13], Istio [9], Bastion [44], and eZTrust [50] regulate communication at individual requests or endpoints. These approaches scale well but cannot capture multi-hop dependencies.
- *Information-flow defenses.* Trapeze [17] and Fabric [40] extend visibility across hops but ignore hop ordering, a distinction essential for separating benign workflows from malicious ones.
- *Multi-hop enforcement.* CloudCover [22], Grewal et al. [28], Meadows et al. [42], and WILLIAM [47] enforce request ordering but route traffic through a centralized controller for decision making, which scales poorly, adds high latency, and creates a single point of failure.

In contrast, eMICRO is the first system to simultaneously provide multi-hop enforcement, hop-ordering preservation, and decentralized scalability. It advances the semantics of microservice security policies beyond prior single-hop access control mechanisms [2, 9, 13, 44, 50] and information-flow defenses that omit

request ordering [17, 40]. Compared to early multi-hop access control systems [22, 28, 34, 42, 47], eMICRO's primary contribution is not new policy expressiveness, but scalable, transparent, decentralized enforcement.

History-based access control extensions. Access control models that incorporate historical context have been studied. The Chinese Wall security policy [21] is an early example, restricting a subject's future accesses based on prior actions to avoid conflicts of interest. Subsequent research extended history-based models to domains like software security, where the execution stack serves as history [16], and mobile application security, where system events provide context [24]. Fong [25] later formalized a general history-based framework.

In this work, we extend history-based access control to microservices, with three components:

(i) Requests. In microservices, the fundamental events to monitor and enforce are requests. Requests typically represent service-to-service communication but the concept extends to other granularities such as pods, IP ranges, or nodes. Similarly, requests may be issued over different protocols (e.g., HTTP, RPC). Thus, a request can be broadly defined as an interaction between two entities in the system, independent of the specific abstraction or protocol.

(ii) Causal dependencies. Requests are often causally related, as one request may trigger others to fulfill users' intent, a common pattern that have been widely studied [33, 35, 37, 41]. Dependencies can be synchronous or asynchronous. In synchronous dependencies, request r_1 triggers request r_2 and cannot complete until r_2 finishes, a pattern common in aggregator services. In asynchronous dependencies, r_1 triggers r_2 but may complete independently, as in queuing systems such as RabbitMQ [14] and Kafka [1].

(iii) Request workflows as history. A request workflow is defined as a sequence of requests in which each request is causally dependent on its predecessor. In practice, the set of valid workflows within a microservice application is finite, since it is constrained by the architecture and service interactions. This set is also substantially smaller than the space of all possible service permutations, making it a practical basis for access control. In our history-based access control model, workflows form the historical context: *a request is permitted only if both the request itself and the causal sequence leading to it are valid.* This ensures that even individually valid requests are denied if they deviate from legitimate workflows.

2.4 Threat Model

Our work focuses on multi-hop attacks. eMICRO is designed to be deployable in both self-managed microservice clusters to enforce application-specific request workflows, and multi-tenant microservice clusters to enforce per-tenant workflows and prevent cross-tenant attacks. We consider attackers who leverage legitimate inter-service requests as stepping stones to access protected services. Such attacks may originate externally through public-facing endpoints or internally through malicious container images. Attackers may perform multi-hop attacks by abusing benign microservice functionality (e.g., server-side request forgery, open-redirection vulnerabilities) or by spawning malicious processes after compromising a service. We assume the underlying platform, namely the host kernel, the eBPF subsystem, and the container runtime, remains

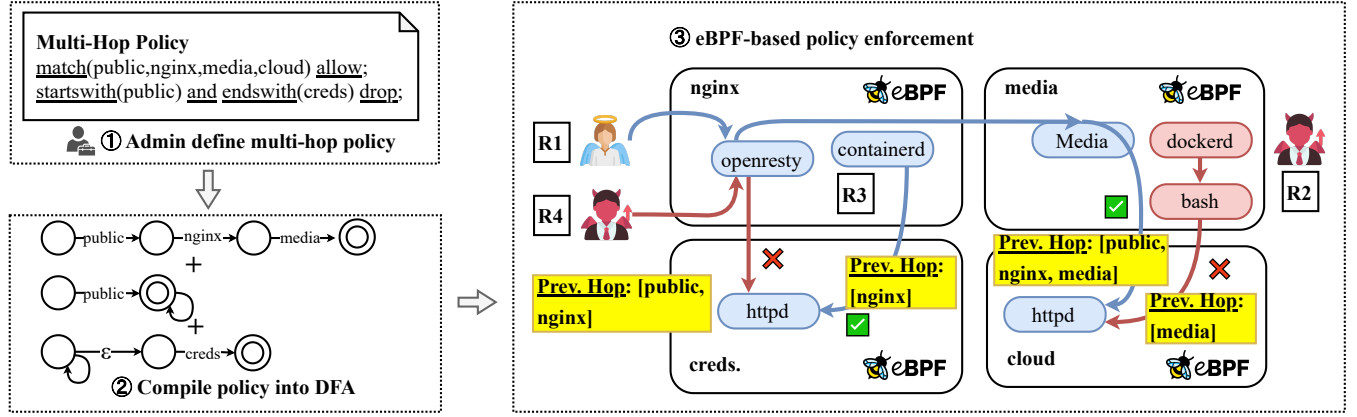


Fig. 2: Overview of eMICRO. Administrators define multi-hop policies (①, § 4.1), which are compiled into a single DFA for efficient label propagation and real-time enforcement (②, § 4.2) and enforced in a distributed manner using eBPF (③, § 4.3). eMICRO blocks multi-hop attacks that compromise services (R2) or leverage benign services (R4) without disrupting legitimate workflows (R1, R3). For example, both R1 and R2 send requests from media to cloud, but only the former follows the intended workflow; similarly, both R3 and R4 send requests from nginx to creds, but the latter is not triggered by a valid request path.

trustworthy, so attackers cannot subvert the kernel or the execution environment of eMICRO’s eBPF programs. Adversaries with kernel-level access could disable the microservice platform entirely, which is beyond the scope of this work and falls under kernel security rather than microservice security. This threat model is aligned with prior work on cloud and network security [17, 22, 28, 34, 47].

3 System Overview

eMICRO protects microservices from multi-hop attacks. Instead of relying on centralized enforcement that cannot scale, eMICRO provides scalable, distributed enforcement with history-based access control. It realizes this through ① an expressive multi-hop policy language, ② compact deterministic finite automaton (DFA) encoding of request sequences, and ③ transparent, application-agnostic, in-kernel enforcement using eBPF.

Fig. 2 presents a system overview. In step ①, administrators define multi-hop security policies using eMICRO’s policy language (§ 4.1), specifying valid request workflows over complete request paths. Typical use cases include preventing unauthorized access, blocking credential leakage, and enforcing control-flow integrity.

Once policies are defined, eMICRO compiles them into a DFA (step ②, § 4.2). Each atomic predicate is translated into a DFA, while composite predicates are constructed using union and intersection operations. DFAs from multiple policies are then merged under deny-by-default semantics using a data-parallel algorithm [30] that combines large policy sets efficiently. The resulting DFA is minimized using Hopcroft’s algorithm [31]. Each DFA state corresponds to a valid request sequence and is assigned a unique label. Requests carry only this label.

In step ③, eMICRO enforces multi-hop policies in a fully distributed and application-agnostic manner (§ 4.3). Each request workflow is assigned an initial DFA label based on its entry point. As requests propagate across services, eMICRO observes network packets and execution contexts at the kernel level and transparently propagates and updates labels across packet headers, processes,

threads, and coroutines. At each hop, label is validated against the DFA transition function and updated to reflect the extended request history. Requests whose labels reach an accepting state continue execution, while those that violate the DFA are blocked. Propagation and enforcement are implemented entirely in the kernel using eBPF-based tracing and execute independently on each service node, eliminating central bottlenecks and enabling low-latency, constant-time checks. To support fast distributed enforcement, the DFA and state mappings are stored in BPF maps and replicated across nodes using a consensus protocol Raft [45], allowing local enforcement without additional network communication.

4 System Design

4.1 Multi-Hop Policy Language

To prevent multi-hop attacks, eMICRO’s policy language provides predicates for pattern matching over complete request paths. As shown in Fig. 3, atomic predicates (match, startswith, endswith, and contains) define regular languages over request paths PH, while predicate composition using and, or, and not corresponds to language intersection, union, and complement. A policy associates a predicate with an action. Enforcement actions (allow, drop) determine whether a request path is accepted or rejected, whereas auxiliary actions (inspect, alert, reset_label, and set_label(x)) introduce side effects without changing path acceptance.

We illustrate the eMICRO’s expressiveness through three multi-hop policies that capture common security requirements:

(i) **Unauthorized access prevention.** The administrator requires that every public request to the cloud database be properly authorized through the public gateway nginx; any request that bypasses nginx (for example, via an accidentally exposed service port) must be dropped. This can be encoded as: startswith(public) and not contains(nginx) and endswith(creds) drop. Because the policy reasons over the full request workflow, it catches violations even when nginx is not the immediate predecessor of creds.


```

<Policy> ::= <Predicate> <Action>
<Action> ::= drop | allow | alert | inspect |
            reset_label | set_label(x)
<Predicate> ::= match(<PH>)
              | startswith(<PH>)
              | endswith(<PH>)
              | contains(<PH>)
              | <Predicate> <Op> <Predicate>
              | not(<Predicate>)
<Op> ::= and | or
<PH> ::= service*

```

Fig. 3: Multi-hop microservices policy language

(ii) **Credentials leakage prevention.** Administrators require the creds service to act as a terminal service that never initiates outbound requests, especially those reaching public. This can be encoded as `startswith(creds) and endswith(public) drop`.

(iii) **Control flow integrity.** Administrators want to enforce exact request flows by enumerating all valid paths (including control-flow variations) and disallowing the rest. The example in Fig. 2 can be implemented as `match(public, nginx, media, cloud) allow; match(nginx, creds) allow`.

4.2 Compiling Policies into DFA

Multi-hop access control requires propagating request-history context, but naively carrying full request sequences and sequentially validating them against policies does not scale.

Naive encoding and its inefficiency. The simplest encoding is to represent the hop sequence as a list of integers. For instance, the workflow A,B can be represented as the list [1, 2], and the workflow A,B,C as [1, 2, 3]. The list must accommodate up to m elements (the maximum depth of any request workflow) and each element must uniquely identify n services. Under this design, the required bits can be derived as follows:

$$b_{naive} = \log_2(n) \times m \quad (1)$$

where b_{naive} denotes the label size in bits, n denotes the number of unique services, and m denotes the maximum request workflow's depth.

Label size is critical as labels are carried with the requests across thousands of services and multiple hops. Inefficient designs can cause unnecessary space overhead. Note that the encoding only needs to represent valid request workflows, since invalid ones are discarded upon sending a malicious request. In contrast, a naive scheme encodes *all* permutations of n services of length m and most of them are invalid request workflows.

DFA encoding. To propagate request sequences efficiently, *EMicro* assigns each valid request sequence a unique label. For example, [A,B] is labeled 1, while [A,B,C] is labeled 2, ensuring that no labels are wasted on invalid workflows. We formalize this approach with a DFA, where each valid sequence corresponds to a state. This compact representation minimizes the number of bits required for label storage, as follows:

$$b_{dfa} = \log_2(s) \quad (2)$$

where b_{dfa} = tag size in bits and s = number of states.

b_{dfa} is substantially smaller than b_{naive} as $s \ll n^m$ in most cases. In the worst case scenario, where all sequences from length of 1 until m are valid, the value of s is as follow:

$$s_{max} = \sum_{k=1}^m n^k = n \frac{n^m - 1}{n - 1} \quad (3)$$

Here, s exceeds n^m , but the bit difference (Δ) is negligible.

$$\begin{aligned} \Delta &= \log_2(s_{max}) - m \log_2(n) \\ &= [\log_2(n) - \log_2(n-1)] - [\log_2(n^m) - \log_2(n^m - 1)] \end{aligned} \quad (4)$$

Even under extreme conditions (e.g., $n=2$, $m=2$), the difference is only 1 bit and shrinks as n and m grow. Therefore, DFA encoding is at worst negligibly less efficient by one bit and more compact than naive encoding most of the time.

Compilation process. The DFA describes the valid request workflows, which is derived from the multi-hop policies. *EMicro* compiles policies into a single DFA through two components. First, a left-to-right parser converts policies into an abstract syntax tree (AST). Second, the AST passes collect all participating services, convert each atomic predicate into a DFA, and merge them into a single DFA.

Policies are transformed into a DFA as follows. Each predicate p denotes a regular language and is therefore expressible as a DFA that accepts a language $\mathcal{L}$ according to Kleene's theorem [36]. The composition $p1$ or $p2$ corresponds to the union $\mathcal{L}' = \mathcal{L}_1 \cup \mathcal{L}_2$, while $p1$ and $p2$ corresponds to the intersection $\mathcal{L}' = \mathcal{L}_1 \cap \mathcal{L}_2$. Actions determine the final DFA semantics: the policy p allow corresponds to $\mathcal{L}$ itself, whereas p deny corresponds to its complement $\mathcal{L}' = \overline{\mathcal{L}}$.

Multiple policies are *merged* into a single DFA as follows.

$$\left(\bigcup_{i=1}^n \mathcal{L}_{a,i} \right) \cap \left(\bigcap_{j=1}^n \mathcal{L}_{d,j} \right) \quad (5)$$

where $\mathcal{L}_{a,i}$ denotes the i -th allow policy and $\mathcal{L}_{d,j}$ denotes the j -th deny policy. This construction requires a request to satisfy at least one allow policy while violating none of the deny policies, thereby enforcing deny-by-default semantics. The multi-hop policies and compiled DFA are illustrated in Fig. 4a.

The union and intersection operations on two DFAs have a time complexity of $O(m \cdot n \cdot k)$, where m and n are the numbers of states in the two DFAs and k is the alphabet size. Our initial experiments show that compiling 1,000 policies into a single DFA takes more than 20 minutes. Hence, *EMicro* applies optimization. In general, union and intersection operations on one pair of DFAs are independent of those on other pairs. These operations are both commutative and associative, making them parallelizable. *EMicro* leverages a data-parallel algorithm [30] such that each process produces a partial result, reducing the number of DFAs by half at each iteration, until a single DFA remains.

To keep the DFA size minimal in the presence of thousands of services and request workflows, we apply *DFA minimization*, which is a classical problem in automata theory, typically solved using Hopcroft's algorithm [31], shown in Algorithm 1. The algorithm groups indistinguishable states that have identical transition behavior by partitioning states such that those in the same partition are equivalent. It begins with two coarse partitions, accepting and non-accepting states, and iteratively refines them until no further refinement is possible. *EMicro* leverages this algorithm by adapting

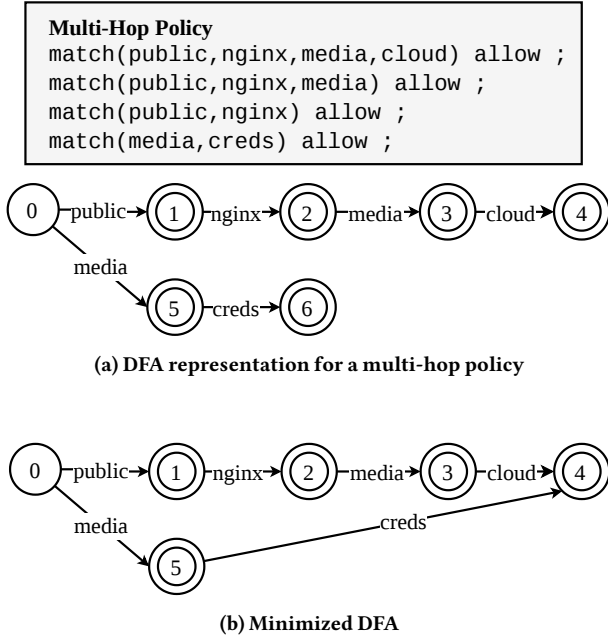


Fig. 4: DFA encoding for multi-hop policies. A six-service deployment with the given set of valid request workflows requires only eight labels to uniquely encode each valid sequence, reduced to five after minimization.

the definitions of states and the transition function to its own model. The minimized DFA, after merging non-distinguishable states, is illustrated in Fig. 4b.

Extending DFA for advanced use case. Although inter-service call cycles are uncommon in conventional microservice architectures, they can arise in practice. Such behaviors do not violate the DFA-based model: bounded service-call loops unroll into finite sequences, while unbounded iterative loops correspond to cycles in the state graph and remain regular by closure of regular languages under Kleene star. If future microservice workflows require unbounded recursion, a more expressive formalism, such as a push-down automaton, could be used.

4.3 Label Propagation and Policy Enforcement

eMicro propagates request labels encoding requests' previous hops as DFA states within and across services to enforce multi-hop policies. A *naive* approach is to attach them to application-level requests (e.g., HTTP headers in HTTP requests, gRPC metadata in RPC), as seen in previous studies and tools (CloudCover [22], Grewal et al. [28], Meadows et al. [42], Jaeger [10]). Handlers can then parse and propagate labels through dedicated or instrumented functions. However, microservices are implemented in diverse environments. For example, Meta reports 16 languages in production [33], while Jaeger and OpenTelemetry [10] maintain instrumentation across 11 languages. Beyond languages, organizations adopt custom dialects (e.g., Meta's Hack, a PHP variant [33]) and a variety of libraries and frameworks. Manually instrumenting each of them is difficult and error-prone, especially under frequent updates, motivating a general solution.

Algorithm 1: Hopcroft Algorithm [31]

```

1:  $P \leftarrow \{F, Q \setminus F\}$  ▷ Current partition
2:  $W \leftarrow \{F, Q \setminus F\}$  ▷ Distinguisher candidate
3: while  $W$  is not empty do
4:   Choose  $A \in W$ 
5:    $W \leftarrow W \setminus \{A\}$ 
6:   for all  $c \in \Sigma$  do
7:      $X \leftarrow \{x \mid x \in Q, \delta[x][c] \in A\}$ 
8:     for all  $Y \in P$  do
9:       if  $X \cap Y = \emptyset \mid Y \setminus X = \emptyset$  then continue
10:      if  $Y \in W$  then ▷  $Y$  has not been used
11:         $W \leftarrow W \setminus Y$ 
12:         $W \leftarrow W \cup \{X \cap Y, Y \setminus X\}$ 
13:      else
14:        if  $|X \cap Y| \leq |Y \setminus X|$  then
15:           $W \leftarrow W \cup \{X \cup Y\}$ 
16:        else
17:           $W \leftarrow W \cup \{Y \setminus X\}$ 

```

To address this challenge, eMicro adopts an in-kernel approach that remains *agnostic* to specific languages, frameworks, libraries, and application-level implementations. The kernel does not directly observe application-level abstractions; instead, it views executing programs as processes and threads, and application-level requests (e.g., HTTP or RPC) as network packets. eMicro propagates labels along these kernel-visible entities. For network packets, eMicro *attaches a custom header after the L4 header*, preserving compatibility with existing L1–L4 functionality such as routing and firewalls. For processes, threads, and coroutines, eMicro taints them with labels stored in BPF maps, keyed by PID, TID, or coroutine context. Beyond agnosticism, this kernel-level design also improves performance by avoiding costly user/kernel context switches.

eBPF-based tracing. Directly modifying the kernel source code is cumbersome and error-prone. eMicro instead leverages eBPF [6], a kernel technology that allows custom programs to run efficiently and safely inside the kernel. Originally designed for packet filtering, eBPF has been extended to support observability, security monitoring and defenses [15, 19, 20], and network performance optimization [8]. eBPF programs are attached to various hook points in the kernel, such as traffic control (TC), express data path (XDP), and kprobes, allowing users to intercept and run custom logic alongside kernel code.

To propagate labels across kernel-visible entities, eMicro leverages three abstractions: network packets, sockets, and execution units. Sockets send and receive packets. Execution units create sockets, consume messages from them, and transmit messages through them. An execution unit is the smallest unit of a program in execution that processes a single request at a time (e.g., a process, thread, or coroutine).

To capture interactions among these entities, eMicro attaches five eBPF programs to various kernel hooks as shown in Fig. 5. Programs (a) and (e) are attached to each service's virtual network interface to propagate labels across services. Programs (b), (c), and (d) are attached to the host kernel, which is shared by all services on that node, to propagate labels within a service. The host kernel

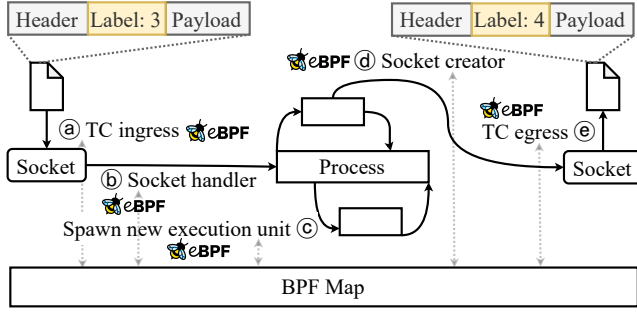


Fig. 5: In-kernel tracing with eBPF. Five hooks are used to propagate labels throughout the request workflows.

is shared between services and it retains visibility into execution units and sockets across all services despite namespace isolation. We describe each eBPF program below:

(a) TC ingress hook. This program runs on each packet arriving at the virtual network interface, parsing the packet header to extract the label, which is present only in the first packet of a flow. Subsequent packets omit the label because they belong to the same request. The program then advances the label to the next state based on the DFA transition function. If the transition is valid, the packet is accepted; otherwise, it is dropped. The receiving socket, identified by its five-tuple, is then tagged with the updated label. Such labels are stored in a BPF map that maps each kernel-visible entity to its current label.

(b) Socket handler. This eBPF program captures the event when an execution unit consumes messages from a socket. On Linux kernel 5.15, it is attached as a kernel return probe (kretprobe) on the `inet_csk_accept` function. The program inspects the internal kernel socket structure to check whether a socket carries a label, and if so, propagates that label to the consuming execution unit.

(c) Spawn new execution unit. This eBPF program captures the event when an execution unit spawns a new one. For multi-process or multi-thread systems, this program is attached as a kernel return probe (kretprobe) on the `clone` syscall. For concurrent systems where the execution unit is a coroutine, this program is attached as a user-space probe (uprobe) on the runtime's coroutine-scheduling function (e.g., `EventLoop` class in Python's `asyncio`, or `runtime.casgstatus` in Go's `goroutine` runtime).

(d) Socket creator. This eBPF program captures the event when an execution unit initiates a new request to another service. On Linux kernel 5.15, it is attached as a kernel probe (probe) on the `tcp_connect` function. This program checks whether the execution unit is already associated with a label. If yes, it propagates the label to the related socket that will send the request.

(e) TC egress hook. This program is triggered whenever a packet leaves a virtual network interface. It checks whether the socket associated with this request carries a label; if so, it attaches the label as a custom header in the packet. Otherwise, it creates a new label representing a single-hop request workflow consisting of the current service.

In deployments with multiple containers on the same physical node, programs (a) and (e) run as one instance per service, each attached to that service's virtual network interface. eMICRO attaches

these programs at service startup and maintains a mapping between network interface identifiers and labels. The host-kernel programs (b), (c), (d) run as a single shared instance per node and observe all containers and processes on that host. Although each container has its own PID namespace, the host kernel sees all processes under globally unique PIDs. eMICRO combines container (cgroup) IDs with PIDs to distinguish processes within each microservice.

Policy enforcement and distribution. eMICRO stores the multi-hop policy, encoded as a DFA, in a BPF map. BPF map is an in-kernel key-value store that naturally represents the DFA transition function: (`previous_state`, `next_hop`) serves as the key and (`next_state`) as the value. For example, the BPF map for Fig. 4b maps (`0`, `public`) to value `1` and (`0`, `media`) to value `5`. This design provides high-performance persistent states shared across eBPF programs and lets each service node consult its local map directly, without extra network requests, enabling distributed and decentralized enforcement. The map can also be updated by user-space programs without recompiling or reloading eBPF code, simplifying policy updates. As a result, *each access control decision becomes a constant-time BPF map lookup*: a request to `svc_a` carrying label `1` is allowed if and only if the BPF map contains a valid transition from (`1`, `svc_a`) to some next state `1'`.

eMICRO distributes policies to participating nodes and maintains consistency using the Raft consensus algorithm [45]. Policy updates are propagated across nodes and committed once a majority of nodes accept them. Failed nodes restart and recover state up to the latest committed update. Nodes exchange periodic heartbeats to ensure leader liveness and to detect and recover missing updates. While policies are replicated across physical nodes, enforcement remains local without additional network overhead.

5 Security Analysis

We analyze the security of eMICRO under the threat model in § 2.4. eMICRO remains resilient against adaptive attackers who know its design and actively attempt to evade, disable, or overload policy enforcement. By enforcing multi-hop policies entirely in kernel space with bounded overhead, eMICRO prevents tracing evasion, detects replayed request histories, and withstands denial-of-service attacks.

Evading eMICRO tracing. An attacker who compromises a service may attempt to evade eMICRO by spawning new processes or threads (e.g., launching a bash shell, escalating privileges, or executing malicious scripts). These attempts fail because eMICRO propagates labels across execution units: child execution units inherit the parent's label. An attacker may further try to escape enforcement by creating a seemingly untainted execution unit with no label. This is not possible: every execution unit has a parent, and if the parent carries no prior history, the child is assigned the initial label `0`, representing a locally initiated request. This does not grant unrestricted privileges: whether requests carrying label `0` are permitted is determined by the administrator's policy, and access is granted only if the resulting request path matches an allowed workflow. While an attacker may still invoke workflows explicitly permitted to originate locally (e.g., routine tasks), they remain confined to that predefined scope and cannot exploit multi-hop interactions to reach unauthorized paths.

Store-and-forward. An attacker may attempt to chain multiple individually valid workflows to bypass the policy. For example, an attacker may bend legitimate workflows through vulnerabilities such as remote code execution or open redirection, or by exploiting permissive suffixes in allowed paths. eMICRO defeats these because each request carries its full causal lineage: even if the bent service is itself allowed to perform certain calls, the resulting end-to-end path (e.g., from an internal sensitive service to an untrusted destination) does not match any administrator-defined workflow and is rejected. More advanced attackers may use indirect store-and-forward: data intended for exfiltration is first staged in an intermediate service and later retrieved through a separate workflow. eMICRO can be extended with additional in-kernel eBPF probes to trace such data-mediated dependencies. For example, filesystem-mediated chains can be reconnected by tracing inode operations: by observing which execution units create, write, and consume each inode, eMICRO propagates labels through the file dependencies and reconstructs the causal chain. We implement this by extending the socket tracing programs (b) and (e) in Fig. 5) from socket file descriptors to user-owned files, attaching probes to the kernel functions `vfs_create`, `vfs_write`, and `vfs_read` to tag the corresponding inode.

Replay attacks. An attacker may attempt to reuse labels from legitimate request workflows, for example, by eavesdropping on traffic or performing man-in-the-middle attacks. eMICRO prevents such replays by binding labels to the execution and communication context in which they were generated: it traces parent-child relationships among execution units and data flows between execution units and network packets, capturing who spawns whom, who sends data, and who receives it. If an attacker injects a valid label into a different execution context, the label becomes inconsistent with the observed execution and communication history, and the request is rejected during policy evaluation.

Denial-of-service attacks. An attacker may attempt to disable eMICRO by flooding it with requests. eMICRO is resilient to such attacks because policy enforcement runs entirely in kernel space and requires no extra network communication. Each policy check executes in constant time, independent of request history length. Thus, eMICRO does not introduce a new bottleneck in the networking stack or virtual interfaces. In practice, any denial-of-service attack sufficient to overwhelm eMICRO would also overwhelm co-located microservices, rather than selectively disabling enforcement while leaving services unprotected.

6 Evaluation

We evaluate eMICRO along three dimensions: (i) effectiveness in preventing multi-hop attacks, (ii) runtime efficiency under realistic microservice workloads, and (iii) scalability of its policy encoding and in-kernel enforcement mechanisms. eMICRO is implemented in approximately 700 lines of C and Python. We conduct end-to-end experiments on a controlled microservice cluster consisting of two physical machines hosting nine virtual machines. Each physical machine is equipped with an Intel Core i7-13700 CPU (2.1 GHz), 32 GB RAM, and runs Ubuntu 20.04. This setup allows us to directly measure enforcement overhead and the impact of in-kernel tracing under concurrent request execution.

Each VM runs the social network application from DeathStar-Bench [26], comprising 27 microservices with 10 replicas, 64 inter-service communication edges, and 32 distinct request workflows. We deploy the application using the benchmark’s provided Docker images, Docker Compose configuration, and Kubernetes deployment scripts, ensuring fidelity to widely used microservice development and deployment practices. We scale the workload generator to issue a total of 12 million requests under sustained concurrency, exercising overlapping request workflows and stressing policy enforcement under load. In addition to benign workloads, we explicitly construct and inject representative multi-hop attacks drawn from real-world attack patterns into the running application to evaluate enforcement correctness under adversarial conditions.

To evaluate eMICRO under diverse networking conditions encountered in modern cloud environments, we repeat experiments across multiple container networking configurations, including Linux bridge networking, Cilium [2], Flannel, and Calico [13]. Beyond online experiments, we evaluate the scalability of eMICRO’s DFA-based policy encoding using production-grade distributed tracing datasets from Uber [37], Alibaba [32, 41], and ByteDance [35]. These datasets contain up to 1.4 million traces collected over seven days, totaling 570 GB of data (shown in Table 4). Each trace is recorded in Jaeger format [10] and captures service-to-service requests linked via explicit parent-child relationships. We reconstruct request workflows from these traces to quantify label space requirements and assess the scalability of eMICRO’s DFA encoding under realistic cloud-scale service topologies.

Our evaluation answers five key research questions.

- (RQ1) *How effective is eMICRO at protecting microservices from multi-hop attacks?* We evaluate eMICRO’s effectiveness in blocking multi-hop attacks, including path-suffix and workflow-bending attacks, and compare its policy coverage over potential attack paths to that of adjacent-hop access control.
- (RQ2) *How efficient is eMICRO at enforcing multi-hop access control in microservices?* We measure end-to-end throughput and request workflow latency, along with the cost of policy compilation and deployment, comparing against prior centralized enforcement approaches.
- (RQ3) *How much space does eMICRO’s DFA-based encoding save compared to naive encoding?* We quantify label-space overhead by comparing naive request-history encoding with eMICRO’s DFA-based encoding on production-scale cloud traces.
- (RQ4) *How scalable are eMICRO’s in-kernel enforcement and label propagation?* We measure the overhead of eBPF-based request tracing, along with policy lookup latency and throughput across hundreds to millions of policies.
- (RQ5) *How much overhead does eMICRO incur to maintain consistency and availability across nodes?* We evaluate eMICRO’s ability to support consistent distributed policy updates and to maintain correct enforcement under node and leader failures.

6.1 RQ1. Defense Effectiveness

We evaluate eMICRO’s defense effectiveness against three representative classes of multi-hop attacks. *First (A1)*, we exploit an

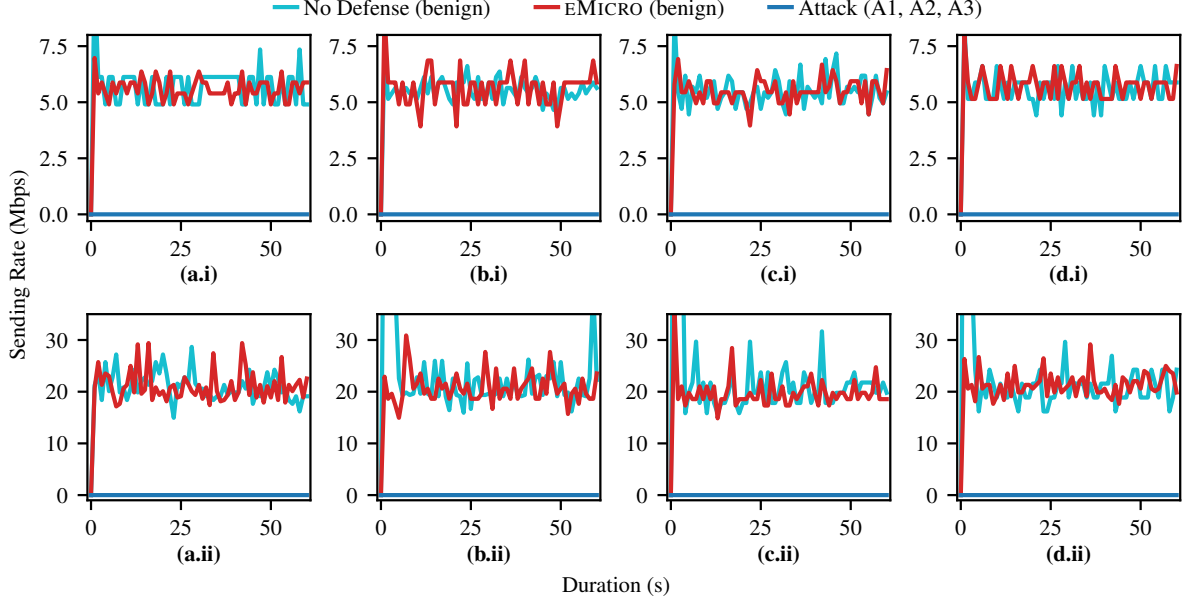


Fig. 6: Service-to-service TCP sending rate over time under benign workloads and representative multi-hop attacks (A1: workflow bending via redirection, A2: RCE-driven lateral movement, A3: malicious container image enumeration). Results are shown across four container networking configurations: (a) Linux bridge, (b) Cilium, (c) Flannel, and (d) Calico. For each configuration, we evaluate two WAN settings: (i) 10 ms latency, 1 ms jitter, 0.1% packet loss, and (ii) 20 ms latency, 5 ms jitter, 0% packet loss.

intentionally introduced open redirection vulnerability in a service without fully compromising it. This attack coerces the vulnerable service into issuing requests to a downstream service that is valid for an adjacent hop but invalid over the complete request path, bending the intended request workflow. *Second* (A2), we exploit an injected remote code execution (RCE) vulnerability in a service, escalate privileges, and initiate connections to other services via SSH. Each individual hop is permitted under adjacent-hop access control, but the resulting end-to-end path violates the intended request workflow. *Third* (A3), we exploit a misconfigured Docker daemon that grants non-privileged users access to the Docker API, allowing them to launch malicious services via `docker run` or `docker exec`. The malicious service performs network enumeration and issues requests that follow valid request-workflow suffixes. However, because they originate locally from the malicious service rather than from the correct preceding hops, they carry an incorrect context (initial label $\emptyset$) instead of a valid state propagated from prior hops, violating the intended request workflow.

We evaluate across four container networking configurations (Linux bridge, Cilium, Flannel, and Calico) and two WAN conditions with latency, jitter, and packet loss of (10 ms, 1 ms, 0.1%) and (20 ms, 5 ms, 0%), yielding eight experimental scenarios. For each attack scenario, we measure under three configurations: (i) a no-defense baseline, (ii) eMICRO enabled under benign workloads, and (iii) eMICRO enabled with the attack active. Defense effectiveness is measured by the number of malicious requests dropped. Performance is measured by the service-to-service TCP sending rate (using `iperf`) since enforcement runs on the network packet path. A sending rate of zero means requests are being dropped.

The results consistently show that eMICRO drops all multi-hop attacks (A1, A2, A3) while preserving all benign requests. It maintains traffic performance, introducing less than 1% reduction in service-to-service TCP sending rate relative to the no-defense baseline (Fig. 6); the impact is not statistically significant.

Multi-hop policy coverage. Beyond blocking concrete attacks, we measure how effectively multi-hop policies reduce the attack surface compared to adjacent-hop access control.

We use the first two days of the Alibaba [32, 41] and Uber [37] traces and reconstruct two types of workflows of lengths 2, 4, and 6. For valid workflows, we recover the causal request chains actually recorded in the dataset by chain requests via their unique identifiers. For invalid workflows, we chain request pairs by matching the destination of one request to the source of the next: each individual hop is valid, yet the chain itself is invalid, indicating the potential attack paths an adversary can exploit. We then measure *policy coverage*, defined as the fraction of all workflows each access control mechanism classifies correctly, i.e., valid workflows permitted and invalid workflows denied.

Table 1 shows that eMICRO maintains 100% coverage at every workflow length, permitting valid workflows and denying invalid ones, because enforcement is bound to the full request history. Adjacent-hop access control, in contrast, inspects only immediate hops and retains no history, so it admits any chained workflow whose individual hops are independently allowed, but the entire path can be malicious. Its coverage therefore degrades as workflows grow longer and the space of attacker-reachable chains expands: at length 6, adjacent-hop correctly classifies only 0.3% of feasible

Table 1: eMICRO’s multi-hop policies coverage

Dataset	# Hops	Adj. Hop (%)	eMICRO (%)
Alibaba [32, 41]	2	99.9	100.0
	4	15.6	100.0
	6	0.3	100.0
Uber [37]	2	2.7	100.0
	4	1.2	100.0
	6	0.1	100.0

workflows. The gap in attack-surface coverage between the two mechanisms grows from $1.01\times$ (2-hop) to $1000\times$ (6-hop), with a geometric mean of $40\times$.

Resilience against store-and-forward evasion. We evaluate a store-and-forward attack in which two individually valid workflows are chained into an invalid end-to-end workflow, by transferring a secret token file to an intermediate service, storing it, and later consuming it through another valid workflow. With data tainting enabled (§ 5), eMICRO blocks all such attempts by propagating labels through inode creation, writes, and reads, so the chain remains visible even when data is persisted to a file. Overhead is minimal: each tracing entry occupies an 8-byte key plus a 2-byte (65K states) or 3-byte (16M states) DFA label, and each tracing operation, including inode reads and writes, incurs less than $2\ \mu\text{s}$.

Takeaway: eMICRO blocks all multi-hop attacks, covering $40\times$ more attack paths on average than adjacent-hop access control, with minimal performance impact on traffic.

6.2 RQ2. System Efficiency

We evaluate eMICRO’s impact on end-to-end request execution using the social network microservice benchmark from DeathStar-Bench [26]. We execute a total of 12 million request workflows with 1,000 concurrent requests and complete the experiments within 12 hours. To assess robustness across deployment environments, we evaluate three configurations: (i) Docker [5] with bridge networking, (ii) a local two-node Kubernetes cluster using Minikube [11] with bridge networking, and (iii) the same Kubernetes setup with Cilium [2] as the container network interface.

System overhead. For each request workflow, we record the end-to-end completion time and construct cumulative distribution functions (CDFs) for runs with and without eMICRO enabled. Fig. 7 shows that eMICRO does not introduce a significant shift in the latency distribution across orchestration platforms and network configurations.

Per-component overhead. To isolate the overhead introduced by individual enforcement components, we measure the additional processing time incurred at each kernel-level hook used by eMICRO. These include TC ingress (`__netif_receive_skb`), TC egress (`__dev_queue_xmit`), and three kernel probes to propagate contexts within services (`inet_csk_accept`, `clone syscall`, `tcp_connect`). Table 2 shows the baseline processing time of each function and the additional cost introduced by eMICRO. In all cases, the added overhead is on the order of microseconds and remains small relative to the original kernel processing cost.

Table 2: Execution time of each kernel function and the overhead added by attaching the eMICRO’s eBPF probe

Kernel Function	Time (μs)	eMICRO Δ (μs)
<code>__netif_receive_skb</code>	12.6	1.4
<code>inet_csk_accept</code>	4.7	2.0
<code>__x64_sys_clone</code>	291	0.7
<code>tcp_connect</code>	51.7	1.9
<code>__dev_queue_xmit</code>	4.1	1.3

Table 3: Compilation time for eMICRO’s policy language under varying numbers of policies, AST depths, and services

# Policy	AST Depth	# Services	Time (s)
100	2	25	0.72
	4		1.25
1000	2	200	106.11
	4		115.01

Table 4: Production cloud trace datasets. We reconstruct request workflows to measure DFA encoding efficiency.

Dataset	# Traces	Duration	Size
Uber [37]	1.4M	7 days	570GB
Alibaba [32, 41]	1M	12 hrs	49GB
ByteDance [35]	1K	N.A. ¹	5MB

Comparison to prior approaches. We compare eMICRO with CloudCover [22], a representative centralized approach. We reimplement CloudCover by manually instrumenting the HTTP server library to propagate request context and deploying a centralized service that infers access control decisions. We measure end-to-end request workflow completion time while varying the number of hops and concurrent requests. As shown in Fig. 8a and Fig. 8b, eMICRO consistently outperforms CloudCover by 35% on average. In addition, eMICRO requires only $1\ \mu\text{s}$ per policy check, compared to 1.6 ms for Kalium [34] and 3 ms for Trapeze. This advantage comes from fully distributed request tracing and enforcement using eBPF, combined with efficient DFA-based policy encoding.

Compiler efficiency. We evaluate the efficiency of eMICRO’s policy compiler by measuring the time required to translate high-level policies into a single enforceable DFA. For various configurations (number of policies, abstract syntax tree depths, and number of services), we measure the total compilation time, including predicate-to-DFA conversion, union and intersection of composite predicates, and merging all policies into one DFA. Table 3 shows that policy compilation time increases with policy count and AST depth but remains well within practical bounds.

Takeaway: eMICRO incurs $<10\ \mu\text{s}$ overhead, preserves microservice orchestration performance, compiles 1000 policies in under 2 minutes, and outperforms CloudCover by 35%.

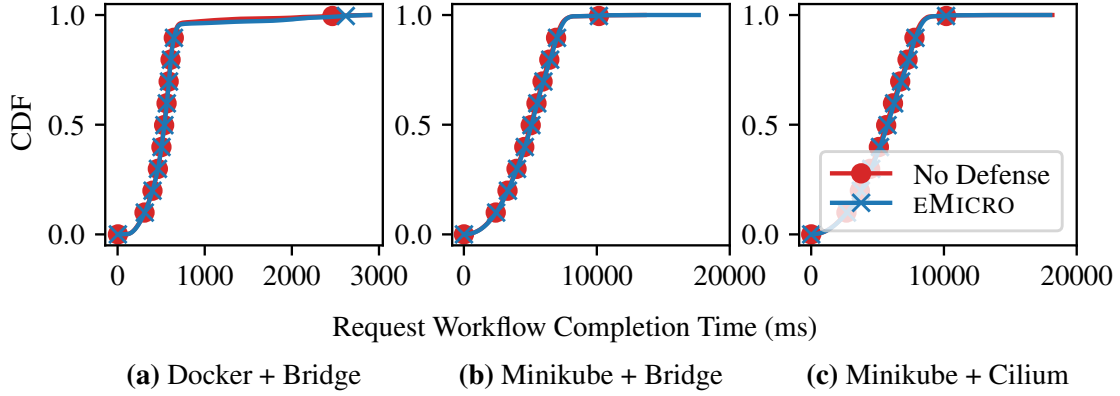


Fig. 7: Cumulative distribution function (CDF) for request workflows completion time in various settings. **EMicro** introduces negligible overhead with no significant CDF shift.

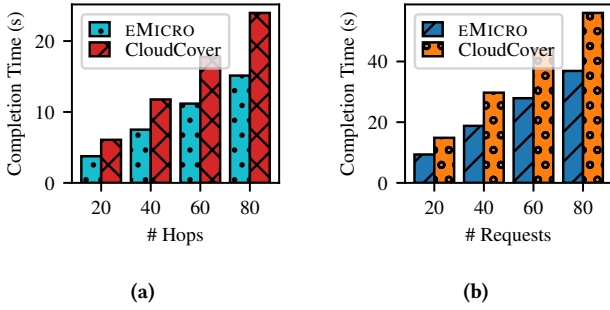


Fig. 8: Comparison of **EMicro** to a major centralized approach across number of (a) hops and (b) requests. **EMicro** requires 0.4s per request and 0.2s per hop, while the centralized approach requires 0.7s and 0.3s, yielding ~35% improvement.

Table 5: Packet header space required to encode request workflows. **EMicro** reduces this overhead by 90% relative to the naive encoding baseline.

Dataset	# Svc	Max hop	# States	Naive (Bits)	DFA (Bits)	$\Delta(\%)$
SocialNetwork [26]	27	5	18	25	5	-80%
Media [26]	33	6	5	36	4	-88%
Uber [37]	2.4K	85	260K	955	18	-98%
Alibaba [32, 41]	45K	31	1.1M	480	21	-95%
ByteDance [35]	3.4K	12	27K	141	15	-89%

6.3 RQ3. Label Efficiency

We compare **EMicro**'s DFA-based strategy against a naive encoding baseline. We use cloud trace datasets from Uber [37], Alibaba [32, 41], and ByteDance [35] as shown in Table 4.

Naive encoding. To calculate the number of bits required in naive encoding, we analyze the number of unique services and the maximum depth of the request workflow, following Eq. (1). Table 5 shows the results. Note that naive encoding can require up to 955 bits (119

¹The information is not mentioned in the partially released data.

bytes) in large-scale microservices, as observed in Uber cloud traces with 2400 services. This exceeds the standard packet headers such as IP (20-60 bytes), TCP (20-60 bytes), and UDP (8 bytes). Using such encoding would significantly reduce system throughput.

DFA encoding. We generate a DFA from the request workflows reconstructed from each of the cloud trace datasets above and compute the required label size using Eq. (2). Table 5 shows that the number of states grows exponentially with the number of services. This is expected since one state represents a distinct request workflow. However, encoding this state space requires at most ~20 bits, a ~90% reduction relative to the baseline.

Takeaway: **EMicro** encodes multi-hop policies for 45K services in just 21-bit labels, a 90% reduction in space overhead.

6.4 RQ4. Scalability of In-Kernel Label Propagation and Policy Enforcement

To evaluate the scalability of **EMicro**'s in-kernel label propagation and policy enforcement, we measure the size of BPF maps and the time taken to build them and perform policy checks, under varying numbers of states and policies. We use three (#States, Key size (bytes), Value size (bytes)) configurations: (256, 2, 1), (65K, 3, 2), and (16M, 4, 3). Here, the value stores a state identifier using the minimum number of bytes needed to encode all states, the key stores the state plus one additional byte for up to 2^8 transitions (the smallest size supported by BPF maps that also ensures policy uniqueness for up to 50M policies in our experiments), and the number of policies affects the number of entries.

BPF map build and insert time. Fig. 9a shows that 50 million multi-hop policies are rebuilt and reinserted into BPF maps in under ~10 seconds, comparable to typical service recovery time such as pulling container images and restarting containers. This confirms that **EMicro**'s recovery cost remains practical at production scale.

BPF map size. Fig. 9b shows that 50 million policies (16 million DFA states) require only 100 MB of space. This efficiency comes from the compact DFA representation of valid request workflows and state minimization: 50 million policies do not translate into

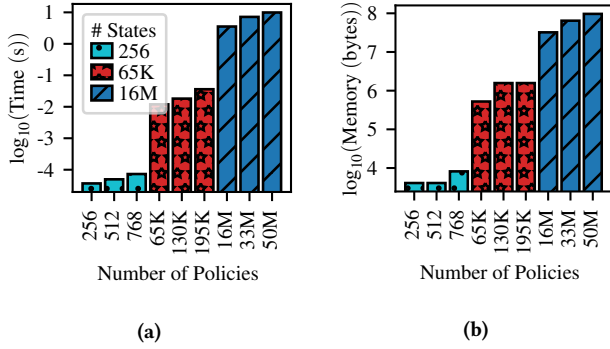


Fig. 9: (a) Time and (b) memory overhead of storing multi-hop policies in BPF maps. eMICRO requires only <10 seconds and 100 MB to rebuild and store 50 million policies.

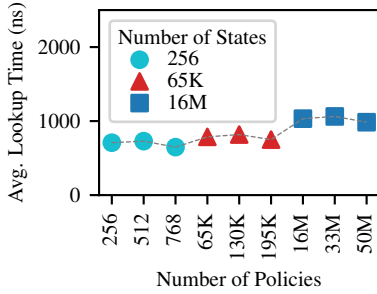


Fig. 10: Mean lookup time across varying policy counts. eMICRO takes only 1 μ s to make an access control decision.

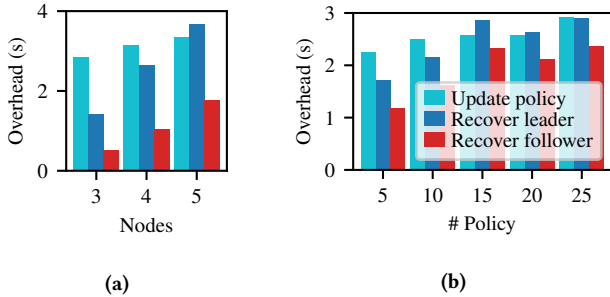


Fig. 11: Time to propagate policy updates and recover from leader and follower failures across varying cluster sizes and policy counts.

50 million BPF map entries. The 16-million-state capacity is large enough to cover every public cloud trace we analyzed; the largest, Alibaba [32, 41], has 45K services and 1.1 million states. Note that the resulting 100 MB of storage for 50 million entries fits easily in modern memory, without requiring complex mechanisms to split policies across nodes or maintain consistency.

Takeaway: eMICRO distributes policy under 4 s and recovers from node failure in an additional 4 s.

Policy lookup time. eMICRO’s decentralized design does not require additional network requests for access control decisions. Fig. 10 shows that the lookup time remains constant for the same number of states. Increasing the state count enlarges the BPF map key, slightly raising hash time by a negligible amount (< 200 ns). Even at 16 million states and 50 million policies, eMICRO resolves an access control decision in just 1 μ s. This scalability stems from the DFA’s deterministic transition and the constant-time complexity of BPF hash map lookup.

Takeaway: eMICRO stores 50 million multi-hop policies in 100 MB, rebuilds them in 10 s, and serves constant-time lookups in under 1 μ s.

6.5 RQ5. Distributed Update and Resilience

We evaluate eMICRO’s ability to support distributed policy updates and recover from node failures using a Raft-replicated policy store. Policy state is represented as key-value entries identical to those enforced in the kernel: each key is a (current state, next hop) tuple and each value is the corresponding next state, so replicated state applies directly to in-kernel BPF maps without translation. We deploy a Raft cluster across multiple nodes, elect a leader, and issue policy updates to that leader, measuring the end-to-end latency to replicate and apply each update. To evaluate fault tolerance, we simulate follower failures by stopping and restarting follower nodes, and leader failures by stopping the leader to trigger re-election. In both cases, we measure the time to restore policy consistency.

Figs. 11a and 11b show that policy updates and recovery from both failure types complete within a few seconds across varying cluster sizes and policy counts, demonstrating that distributed enforcement state can be maintained with low operational overhead.

7 Discussion

Compatibility with connection pooling and multiplexing.

eMICRO operates in a protocol-agnostic manner. When an L4 connection carries only one request (i.e., neither pooled nor reused), the connection itself suffices as the request identifier (e.g., a socket file descriptor or 5-tuple). If connections are pooled or reused, the same identifier can be used but must be refreshed upon reuse. In deployments that multiplex requests over a single L4 connection (e.g., HTTP/2, HTTP/3), eMICRO requires a per-request identifier that distinguishes concurrent streams within the connection. For example, it may be defined as a tuple of the L4 connection identifier and the stream ID (e.g., HTTP/2 stream ID). More generally, eMICRO can use any request identifier observable from the eBPF program at the relevant granularity: socket-level for non-multiplexed traffic, stream-level for multiplexed traffic.

Compatibility with other networking stacks. We prototype eMICRO with eBPF programs attached to TC and XDP because these mechanisms are widely deployed in microservice networks (e.g., Cilium, Calico) and provide low-latency, line-rate processing in the kernel networking stack. However, eMICRO’s design is not limited to the kernel networking stack itself. In environments that use kernel-bypass networking stacks (e.g., DPDK), the same processing logic (label parsing, DFA transitions, enforcement) can be

ported. For example, packet parsing can be implemented as a DPDK user-space program that extracts and inserts DFA labels. It can access the DFA state and transition maps through shared memory (e.g., pinned BPF maps). Execution unit tracing remains in eBPF programs in the host kernel, since it relies on kernel events (e.g., socket operations, syscalls) that DPDK does not bypass.

Propagating labels in asynchronous workflows. eMICRO naturally supports both synchronous and asynchronous execution because it tracks causality through interactions between execution units and network connections. As long as the DFA label is propagated through these channels, eMICRO preserves it correctly. When request workflows are mediated by a broker, queue, or shared storage, the causal chain breaks at this boundary. This is an inherent limitation of all history-based systems, since no observable signal remains to reconstruct the missing dependency. In such cases, eMICRO can be extended with lightweight instrumentation at the broker, queue, or shared storage to attach and propagate labels, preserving causality across these boundaries.

Policy distribution. Although our evaluation shows that it is possible for every node to hold the complete 50 million multi-hop policies for large microservice deployments, there may be times when the policies are so large that they must be partitioned across nodes. A distributed scheme would ensure that each node holds at least the policies for the services assigned to it, while keeping partitions reconstructible in the event of node failure. Designing such a scheme is beyond the scope of this paper and left for future work.

8 Conclusion

eMICRO is a protocol-agnostic, scalable multi-hop access control system for microservices. It compiles policies into a compact DFA in which each request maps to a state, enabling constant-time checks. This state is propagated by tracing request execution across multi-language microservices using eBPF probes on kernel events and entities. Our evaluations on widely-established benchmarks and production cloud traces show that eMICRO adds negligible performance overhead and scales to large deployments.

We see several promising directions for future work. First, policy checks could be offloaded or performed preemptively in the network using P4-programmable switches, enabling line-rate enforcement and reducing host overhead in multi-node deployments. Second, eMICRO's tracing mechanism can be extended to microservice performance profiling, providing zero-modification instrumentation across heterogeneous languages and protocols. Third, eMICRO can be extended to support more expressive policy models beyond regular languages, such as pushdown automata (PDA), to capture more complex request workflow dependencies.

Ethical Considerations

Microservices support critical modern systems, and their misuse has led to publicly reported catastrophic failures. By enforcing multi-hop request workflows rather than only adjacent hops, eMICRO provides a more secure runtime environment and reduces the likelihood of such failures through fine-grained, end-to-end enforcement using eBPF and compact DFA representations.

Throughout its development, eMICRO relied exclusively on ethically sourced, publicly available datasets. All experiments were

performed in isolated infrastructure under our control, without interacting with or impacting external systems. We recognize that publishing detailed security mechanisms can inform both defenders and adversaries; however, we believe that transparency and open evaluation are essential for scientific integrity and for enabling the community to build safer, more resilient systems

Open Science

We provide an implementation of eMICRO along with documentation to support future research building on our work. The artifact is available at: <https://github.com/peng-gao-lab/emicro>.

Acknowledgement

We would like to thank the anonymous reviewers and our shepherd for their constructive comments. This work is supported in part by the National Science Foundation under grant 2442171 and the Google Academic Research Award. Any opinions, findings, and conclusions made in this paper are those of the authors and do not necessarily reflect the views of the funding agencies. We acknowledge the use of large language models for polishing the manuscript, including grammar, punctuation, and minor edits. All experimental results, design choices, and the main contributions of the paper remain the sole responsibility of the authors.

References

- [1] [n. d.]. Apache Kafka. <https://kafka.apache.org>. [Accessed 07-08-2025].
- [2] [n. d.]. Cilium - Cloud Native, eBPF-based Networking, Observability, and Security. <https://cilium.io>. [Accessed 28-07-2025].
- [3] [n. d.]. Containerd. <https://containerd.io>. [Accessed 08-08-2025].
- [4] [n. d.]. CWE - CWE-601: URL Redirection to Untrusted Site (Open Redirect) (4.17). <https://cwe.mitre.org/data/definitions/601.html>. [Accessed 06-08-2025].
- [5] [n. d.]. Docker: Accelerated Container Application Development. <https://www.docker.com>. [Accessed 28-07-2025].
- [6] [n. d.]. eBPF - Introduction, Tutorials & Community Resources. <https://ebpf.io>. [Accessed 08-08-2025].
- [7] [n. d.]. Enterprise Tactics | MITRE ATT&CK. <https://attack.mitre.org/tactics/enterprise/>. [Accessed 06-08-2025].
- [8] [n. d.]. A High Performance Layer 4 Load Balancer. <https://github.com/facebookincubator/katran>. [Accessed 08-08-2025].
- [9] [n. d.]. Istio. <https://istio.io>. [Accessed 29-07-2025].
- [10] [n. d.]. Jaeger: Open Source, Distributed Tracing Platform. <https://www.jaegertracing.io>. [Accessed 28-07-2025].
- [11] [n. d.]. Minikube. <https://minikube.sigs.k8s.io>. [Accessed 28-07-2025].
- [12] [n. d.]. Production-Grade Container Orchestration. <https://kubernetes.io>. [Accessed 08-08-2025].
- [13] [n. d.]. Project Calico | Tigera - Creator of Calico. <https://www.tigera.io/project-calico/>. [Accessed 29-07-2025].
- [14] [n. d.]. RabbitMQ: One Broker to Queue Them All. <https://www.rabbitmq.com>. [Accessed 07-08-2025].
- [15] [n. d.]. Tetragon - eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io>. [Accessed 08-08-2025].
- [16] Martin Abadi and Cédric Fournet. 2003. Access Control based on Execution History. In *10th Annual Network and Distributed System Security Symposium (NDSS '03)*. 107–121.
- [17] Kalev Alpernas, Cormac Flanagan, Sadjad Fouladi, Leonid Ryzhyk, Mooly Sagiv, Thomas Schmitz, and Keith Winstein. 2018. Secure Serverless Computing Using Dynamic Information Flow Control. *Proc. ACM Program. Lang.* 2, Article 118 (Oct. 2018), 26 pages.
- [18] Osama Bajaber, Bo Ji, and Peng Gao. 2024. P4Control: Line-Rate Cross-Host Attack Prevention via In-Network Information Flow Control Enabled by Programmable Switches and eBPF. In *2024 IEEE Symposium on Security and Privacy (SP '24)*. 4610–4628.
- [19] Osama Bajaber, Bo Ji, and Peng Gao. 2026. NetCap: Data-Plane Capability-Based Defense Against Token Theft in Network Access. In *Proceedings of the Network and Distributed System Security (NDSS '26) Symposium*.
- [20] William Blair, Frederico Araujo, Teryl Taylor, and Jiyong Jang. 2024. Automated Synthesis of Effect Graph Policies for Microservice-Aware Stateful System Call Specialization. In *2024 IEEE Symposium on Security and Privacy (SP '24)*. 4554–4572.

- [21] D.F.C. Brewer and M.J. Nash. 1989. The Chinese Wall Security Policy. In *1989 IEEE Symposium on Security and Privacy (SP '89)*. 206–214.
- [22] Dalton A. Brucker-Hahn, Wang Feng, Shanchao Li, Matthew Petillo, Alexandru G. Bardas, Drew Davidson, and Yuede Ji. 2024. CloudCover: Enforcement of Multi-Hop Network Connections in Microservice Deployments. In *2024 Annual Computer Security Applications Conference (ACSAC '24)*. 1186–1202.
- [23] Xutong Chen, Hassaan Irshad, Yan Chen, Ashish Gehani, and Vinod Yegneswaran. 2021. CLARION: Sound and Clear Provenance Tracking for Microservice Deployments. In *30th USENIX Security Symposium (USENIX Security '21)*. 3989–4006.
- [24] Guy Edjlali, Anurag Acharya, and Vipin Chaudhary. 1998. History-based Access Control For Mobile Code. In *Proceedings of the 5th ACM Conference on Computer and Communications Security (CCS '98)*. 38–48.
- [25] Philip WL Fong. 2004. Access Control by Tracking Shallow Execution History. In *2004 IEEE Symposium on Security and Privacy (SP '04)*. 43–55.
- [26] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. 3–18.
- [27] Seyedhamed Ghavamnia, Tapti Palit, Azzedine Benameur, and Michalis Polychronakis. 2020. Confine: Automated System Call Policy Generation for Container Attack Surface Reduction. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID '20)*. 443–458.
- [28] Karuna Grewal, P. Brighten Godfrey, and Justin Hsu. 2023. Expressive Policies For Microservice Networks. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets '23)*. 280–286.
- [29] Wajih Ul Hassan, Lemay Aguse, Nuraini Aguse, Adam Bates, and Thomas Moyer. 2018. Towards Scalable Cluster Auditing through Grammatical Inference over Provenance Graphs. In *2018 Network and Distributed Systems Security Symposium (NDSS '18)*.
- [30] W. Daniel Hillis and Guy L. Steele. 1986. Data Parallel Algorithms. *Commun. ACM* 29, 12 (Dec. 1986), 1170–1183.
- [31] John Hopcroft. 1971. An $n \log n$ Algorithm for Minimizing States in a Finite Automaton. In *1971 Theory of Machines and Computations*. 189–196.
- [32] Darby Huye. 2024. Alibaba Distributed Traces (2022). <https://doi.org/10.7910/DVN/T53HGF>.
- [33] Darby Huye, Yuri Shkuro, and Raja R. Sambasivan. 2023. Lifting the Veil on Meta's Microservice Architecture: Analyses of Topology and Request Workflows. In *2023 USENIX Annual Technical Conference (USENIX ATC '23)*. 419–432.
- [34] Deepak Sirone Jegan, Liang Wang, Siddhant Bhagat, and Michael Swift. 2023. Guarding Serverless Applications with Kalium. In *32nd USENIX Security Symposium (USENIX Security '23)*. 4087–4104.
- [35] Rathinaraja Jeyaraj, Anandkumar Balasubramaniam, Ajay Kumara M.A., Nadra Guizani, and Anand Paul. 2023. Resource Management in Cloud and Cloud-influenced Technologies for Internet of Things Applications. *ACM Comput. Surv.* 55, 12, Article 242 (March 2023), 37 pages.
- [36] Stephen Cole Kleene. 1956. 1956 Representation of Events in Nerve Nets and Finite Automata. In *Automata Studies*. 3–41.
- [37] I-Ting Angelina Lee, Zhizhou Zhang, Abhishek Parwal, and Milind Chabbi. 2024. The Tale of Errors in Microservices. *Proc. ACM Meas. Anal. Comput. Syst.* 8, 3 (Dec. 2024), 36 pages.
- [38] Xing Li, Yan Chen, and Zhiqiang Lin. 2019. Towards Automated Inter-Service Authorization for Microservice Applications. In *Proceedings of the ACM SIGCOMM 2019 Conference Posters and Demos (SIGCOMM Posters and Demos '19)*. 3–5.
- [39] Xing Li, Yan Chen, Zhiqiang Lin, Xiao Wang, and Jim Hao Chen. 2021. Automatic Policy Generation for Inter-Service Access Control of Microservices. In *30th USENIX Security Symposium (USENIX Security '21)*. 3971–3988.
- [40] Jed Liu, Owen Arden, Michael D. George, Andrew C. Myers, Toby Murray, Andrei Sabelfeld, and Lujo Bauer. 2017. Fabric: Building Open Distributed Systems Securely by Construction. *J. Comput. Secur.* 25, 4–5 (Jan. 2017), 367–426.
- [41] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2022. The Power of Prediction: Microservice Auto Scaling via Workload Learning. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '22)*. 355–369.
- [42] Catherine Meadows, Sena Hounsinnou, Timothy Wood, and Gedare Bloom. 2023. Sidecar-based Path-aware Security for Microservices. In *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies (SACMAT '23)*. 157–162.
- [43] Matt Muir and Andy Giron. 2024. Threat Actors Leverage Docker Swarm and Kubernetes to Mine Cryptocurrency at Scale. <https://securitylabs.datadoghq.com/articles/threat-actors-leveraging-docker-swarm-kubernetes-mine-cryptocurrency>. [Accessed 04-08-2025].
- [44] Jaehyun Nam, Seungsoo Lee, Hyunmin Seo, Phil Porras, Vinod Yegneswaran, and Seungwon Shin. 2020. BASTION: A Security Enforcement Network Stack for Container Networks. In *2020 USENIX Annual Technical Conference (USENIX ATC '20)*. 81–95.
- [45] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC '14)*. 305–319.
- [46] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. 2017. Practical Whole-System Provenance Capture. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*. 405–418.
- [47] Arnav Sankaran, Pubali Datta, and Adam Bates. 2020. Workflow Integration Alleviates Identity and Access Management in Serverless Computing. In *Proceedings of the 36th Annual Computer Security Applications Conference (ACSAC '20)*. 496–509.
- [48] Rui Shu, Xiaohui Gu, and William Enck. 2017. A Study of Security Vulnerabilities on Docker Hub. In *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY '17)*. 269–280.
- [49] Shaowen Xu, Qihang Zhou, Heqing Huang, Xiaoqi Jia, Haichao Du, Yang Chen, and Yamin Xie. 2023. Log2Policy: An Approach to Generate Fine-Grained Access Control Rules for Microservices from Scratch. In *Proceedings of the 39th Annual Computer Security Applications Conference (ACSAC '23)*. 229–240.
- [50] Zirak Zaheer, Hyunseok Chang, Sarit Mukherjee, and Jacobus Van der Merwe. 2019. eZTrust: Network-Independent Zero-Trust Perimeterization for Microservices. In *Proceedings of the 2019 ACM Symposium on SDN Research (SOSR '19)*. 49–61.